Module 3: Packages, Exception Handling and Multithreading

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai Campus

premanand.s@vit.ac.in

March 2, 2026

Learning Progression in Java

- Java programming is taught in a **progressive and layered manner**.
- Each module addresses a specific level of software development maturity.
- Module 3 is introduced **after** fundamentals and OOP design are established.

Progression:

$$\text{Module 1} \rightarrow \text{Module 2} \rightarrow \text{Module 3}$$

What We Achieved in Module 1

- Understanding the **Java programming model**
- JVM, bytecode, and program execution flow
- Basic programming constructs:
  - Data types, variables, operators
  - Control and looping constructs
  - Arrays and strings
- Writing **syntactically correct** Java programs

Limitation: Programs are small, sequential, and not scalable.

What Module 2 Added

- Object-oriented design using:
  - Classes and objects
  - Inheritance and polymorphism
  - Abstract classes and interfaces
- Code reusability and modularity
- Real-world problem modeling

Limitation:

- No standard project organization
- No mechanism to handle runtime failures
- Programs execute in a single thread

Why Module 3 Is Required Now

- Real-world Java applications are:
  - Large
  - Error-prone
  - Concurrent
- Object-oriented design alone is **not sufficient**
- Application-level concerns must be addressed

Module 3 bridges the gap between:

$$\textit{Object-Oriented Design} \rightarrow \textit{Industrial Java Applications}$$

Problems Without Module 3

- Large number of classes become difficult to manage
- Programs crash due to unhandled runtime exceptions
- Inefficient CPU utilization due to sequential execution
- Shared data inconsistencies in concurrent environments

How Module 3 Solves These Problems

- **Packages**
  - Creating and accessing packages
  - Sub-packages and hierarchical organization
- **Exception Handling**
  - Types of exceptions (Checked and Unchecked)
  - Control flow in exceptions
  - Use of `try`, `catch`, `finally`
  - Use of `throw` and `throws`
  - User-defined exceptions
- **Multithreading**
  - Basics of threads
  - Defining threads using `Runnable` interface
  - Multiple threads execution
  - Inter-thread communication
  - Synchronization for data consistency

Why Module 3 Comes After Module 2

- Packages require understanding of classes
- Exceptions propagate through method calls
- Threads operate on objects and shared resources

Hence:

Module 3 logically depends on Modules 1 and 2

Outcome of Module 3

- Ability to build:
  - Structured Java projects
  - Robust and error-resilient applications
  - Concurrent and high-performance systems
- Readiness for:
  - Industry-level Java development
  - Advanced Java frameworks

Why Do We Need Packages?

Why Do We Need Packages?

What is a Package?

Definition

A **package** in Java is a collection of related classes and interfaces grouped together under a common namespace.

- Acts like a **folder** in a file system
- Provides a **namespace** to avoid naming conflicts
- Helps in **logical organization** of code
- Improves **maintainability and reusability**

Simple Analogy

Classes : Doctors
Package : Department

Why Do We Need Packages?

Without Packages:

- All classes go into the default package
- Naming conflicts may occur
- Difficult to maintain large projects
- No proper modularization
- Limited control over accessibility (security issues)

With Packages:

- Logical grouping of related classes
- Avoid naming conflicts
- Better access control
- Code reusability
- Improved maintainability

Syntax of Package in Java

Basic Syntax:

Package Declaration

```
package package_name;
```

Example:

Example

```
package hospital;

public class Doctor {
 void display() {
  System.out.println("Doctor Class");
 }
}
```

Step 1: Opening Command Prompt

Action:

- Press `Windows + R`
- Type `cmd`
- Press Enter

Initial Location:

```
C:\Users\username>
```

Learning:

- This is the user home directory.
- All navigation starts from here.

Step 2: Checking Available Folders

Command Used:

```
dir
```

Purpose:

- Lists all folders inside current directory
- Helps locate Desktop folder

Observation:

- Desktop was inside `OneDrive`

Step 3: Navigating to Desktop

Command Used:

```
cd OneDrive\Desktop
```

Now Location Becomes:

```
C:\Users\username\OneDrive\Desktop>
```

Learning:

- `cd` means change directory
- Windows may store Desktop inside OneDrive

Step 4: Creating Project Folder

Command Used:

```
mkdir JavaPackageDemo
cd JavaPackageDemo
```

Learning:

- `mkdir` creates new folder
- Always work inside a dedicated project directory

Step 5: Checking Java Installation

Command Used:

```
java -version
```

Error Observed:

```
'java' is not recognized...
```

Learning:

- Java was not installed
- JDK installation is mandatory

Step 6: Installing JDK and Verifying

Download JDK From:

- Website: `https://adoptium.net`
- Version Used: **OpenJDK 21 (LTS)**
- Platform: Windows x64 (.msi installer)

After Installation, Verify Using:

```
java -version
javac -version
```

Learning:

- `java` runs the JVM (Java Virtual Machine)
- `javac` compiles .java into .class files
- JDK = JVM + Compiler + Development Tools

Step 7: Creating Package Structure

Command Used:

```
mkdir student
```

Learning:

- Package name must match folder name
- Java follows directory-based organization

Creating Java Files Using Notepad

Step 1: Create Package Class (Student.java)

```
notepad student\Student.java
```

- Notepad opens automatically.
- Paste the Student.java code.
- Click File → Save.
- Close Notepad.

Step 2: Create Main Class (MainApp.java)

```
notepad MainApp.java
```

- Notepad opens again.
- Paste the MainApp.java code.
- Click File → Save.
- Close Notepad.

Student.java (Inside Package)

```
package student;
public class Student {
 String name;
 int rollNo;
 double marks;
 public Student(String name, int rollNo, double marks) {
  this.name = name;
  this.rollNo = rollNo;
  this.marks = marks;
 }

 public void displayDetails() {
  System.out.println("Student Name: " + name);
  System.out.println("Roll Number: " + rollNo);
  System.out.println("Marks: " + marks);
 }  }
```

MainApp.java (Accessing Package)

```
import student.Student;

public class MainApp {
 public static void main(String[] args) {

  Student s1 = new Student("Arun", 101, 89.5);
  s1.displayDetails();
 }
}
```

Step 8: Compilation Process

Compile Command:

```
javac student\Student.java MainApp.java
```

Learning:

- Compiler generates `.class` files
- Compilation must be done from project root

Step 9: Running the Program

Run Command:

```
java MainApp
```

Execution Flow:

- JVM loads MainApp.class
- Accesses Student.class from student folder
- Program executes successfully

What Did We Learn from the Package Demo?

- **Packages are not just syntax**
  - `package student;` is linked to folder structure and JVM
- **Folder structure must match package name**
  - Package name = Directory name
- **Import is required for cross-package access**
  - `import student.Student;`
- **Access modifiers matter**
  - Class must be `public` to access outside package
- **Compilation location is important**
  - Compile from project root directory
- **Understanding JVM Class Loading**
  - JVM searches for `student/Student.class`
- **Big Takeaway**
  - Packages enable modular, maintainable, industry-level Java applications

Access Modifiers Across Packages

Access in Java depends on:

- Access modifier (`private`, default, `protected`, `public`)
- Whether classes are in same or different package
- Whether inheritance is involved

Key Idea: Access control works across

- Package boundaries
- Inheritance hierarchy

Access Control Comparison Table

Modifier	Class Itself	Same Package	Diff. Pkg (Subclass)	Diff. Pkg (Non-Subclass)
private	✓	×	×	×
default	✓	✓	×	×
protected	✓	✓	✓	×
public	✓	✓	✓	✓

Demo Structure

Project Structure:

```
project/
 pack1/
      A.java
 pack2/
      B.java
```

pack1/A.java

```
package pack1;

public class A {

 private int privateVar = 10;
 int defaultVar = 20;
 protected int protectedVar = 30;
 public int publicVar = 40;

 public void display() {
  System.out.println(privateVar);
  System.out.println(defaultVar);
  System.out.println(protectedVar);
  System.out.println(publicVar);
 }
}
```

pack2/B.java (Without Inheritance)

```
package pack2;

import pack1.A;
public class B {
 public static void main(String[] args) {

  A obj = new A();

  // System.out.println(obj.privateVar);   // Error
  // System.out.println(obj.defaultVar);   // Error
  // System.out.println(obj.protectedVar); // Error
  System.out.println(obj.publicVar);       // Works
 }
}
```

Conclusion: Only public is accessible across packages (no inheritance).

pack2/B.java (With Inheritance)

```
package pack2;
import pack1.A;
public class B extends A {
 public static void main(String[] args) {
  B obj = new B();
  // System.out.println(obj.privateVar);   // Error
  // System.out.println(obj.defaultVar);   // Error
  System.out.println(obj.protectedVar);    // Works
  System.out.println(obj.publicVar);       // Works
 }
}
```

Conclusion:

- protected works with inheritance
- public works always

Important Observations

- default access is package-level only
- protected allows subclass access across packages
- private is strictly within the same class
- public has no restriction
- Sub-package is NOT same as parent package

Conceptual Understanding Questions

- Why is default not accessible in different package?
- Why does protected require inheritance?
- What happens if class A is not declared public?
- Is sub-package considered same package?

Key Takeaway:
Access modifiers define visibility across both package boundaries and inheritance hierarchy.

Access Modifiers Across Packages – Comparison

1. **Same Package**

```
package pack1;
A obj = new A();
obj.x; // private
obj.y; // default
obj.z; // protected
obj.w; // public
```

Access Modifiers Across Packages – Comparison

1. Same Package

```
package pack1;
A obj = new A();
obj.x; // private
obj.y; // default
obj.z; // protected
obj.w; // public
```

2. Different Package (No Inheritance)

```
package pack2;
import pack1.A;
A obj = new A();
obj.y;
obj.z;
obj.w;
```

Works:

- y, z, w

Access Modifiers Across Packages – Comparison

1. Same Package

```
package pack1;
A obj = new A();
obj.x; // private
obj.y; // default
obj.z; // protected
obj.w; // public
```

Works:

- y, z, w

2. Different Package (No Inheritance)

```
package pack2;
import pack1.A;
A obj = new A();
obj.y;
obj.z;
obj.w;
```

Works:

- w only

3. Different Package (With Inheritance)

```
package pack2;
import pack1.A;
class B extends A {
 B obj = new B();
 obj.z;
 obj.w;
}
```

Access Modifiers Across Packages – Comparison

1. Same Package

```
package pack1;
A obj = new A();
obj.x; // private
obj.y; // default
obj.z; // protected
obj.w; // public
```

Works:

- y, z, w

2. Different Package (No Inheritance)

```
package pack2;
import pack1.A;
A obj = new A();
obj.y;
obj.z;
obj.w;
```

Works:

- w only

3. Different Package (With Inheritance)

```
package pack2;
import pack1.A;
class B extends A {
 B obj = new B();
 obj.z;
 obj.w;
}
```

Works:

- z, w

Access Modifiers Across Packages – Comparison

1. Same Package

```
package pack1;
A obj = new A();
obj.x; // private
obj.y; // default
obj.z; // protected
obj.w; // public
```

Works:

- y, z, w

2. Different Package (No Inheritance)

```
package pack2;
import pack1.A;
A obj = new A();
obj.y;
obj.z;
obj.w;
```

Works:

- w only

3. Different Package (With Inheritance)

```
package pack2;
import pack1.A;
class B extends A {
 B obj = new B();
 obj.z;
 obj.w;
}
```

Works:

- z, w

- private → Only inside class
- default → Same package only
- protected → Same package + Subclass

Thank You!

Stay Connected

Premanand S

Email:	premanand.s@vit.ac.in
Phone:	+91-7358679961
LinkedIn:	linkedin.com/in/premsanand
Instagram:	instagram.com/premsanand
WhatsApp Channel:	anandsDataX
Google Scholar:	Google Scholar Profile
GitHub:	github.com/anandprems